UNIVERSITÀ DEGLI STUDI DI NAPOLI FEDERICO II

Scuola Politecnica e delle Scienze di Base

Dipartimento di Ingegneria Elettrica e delle Tecnologie dell’Informazione

BugBoard26

Progetto di Ingegneria del Software

Documentazione di progetto

Requisiti, design del sistema, sviluppo, testing e usabilità

Gruppo e autori: da completare

Matricole: da completare

21 ottobre 2026

Indice

1 Glossario....4
2 Introduzione....5
2.1 Ambito del sistema....5
2.2 Decisioni di ambito....5
2.3 Organizzazione della relazione....5
3 Raccolta e analisi dei requisiti....6
3.1 Utenti target e personas....6
3.2 Attori e responsabilità....6
3.3 Scenari principali....6
4 Specifica dei requisiti....7
4.1 Diagramma dei casi di uso....7
4.2 Requisiti funzionali....8
4.3 Requisiti non funzionali....9
4.4 Requisiti di dominio....10
4.5 Casi di uso nel modello di Cockburn....11
5 Progettazione architetturale e scelte tecnologiche....20
5.1 Descrizione della architettura....20
5.2 Criteri di design e motivazioni....21
5.3 Tecnologie adottate....21
5.4 Persistenza dei dati....22
5.5 Inizializzazione e conservazione dei dati....23
5.6 Autenticazione sessione e autorizzazione....24
5.7 Gestione delle immagini e delle transazioni....24
6 Design della interfaccia utente....25
6.1 Colori e tipografia....25
6.2 Accesso e scelta del progetto....25
6.3 Dashboard e filtri....27
6.4 Creazione e prevenzione degli errori....29
6.5 Dettaglio commenti e assegnazione....30
6.6 Feedback e notifiche....31
6.7 Amministrazione e limiti da verificare....32
7 Moduli e diagrammi di design....33
7.1 Design del backend....33
7.2 Design del frontend....34
7.3 API e formati delle risposte....34
7.4 Sequenza di autenticazione....36
7.5 Sequenza di creazione con immagine....36
7.6 Sequenza di risoluzione....37

8 Build e avvio del sistema....38
8.1 Backend e Dockerfile....38
8.2 Build del frontend....38
8.3 Docker Compose e configurazione....38
8.4 Configurazione e comandi....38
9 Strumento di versionamento....39
9.1 Organizzazione del lavoro....39
9.2 Struttura del repository....39
10 Qualità del codice....40
10.1 Controlli effettivamente eseguiti....40
10.2 Scelte di qualità osservabili....40
10.3 Verifiche ancora da completare....40
11 Testing automatico....41
11.1 Strategia e ambiente....41
11.2 Comandi per ripetere i test....41
11.3 Test selezionati del cambio di stato....42
11.4 Test selezionati delle immagini....43
11.5 Test frontend....44
11.6 Tracciabilità e copertura funzionale....44
12 Valutazione della usabilità....45
12.1 Review sistematica con checklist....45
12.2 Piano delle prove con utenti....46
12.3 Dati da raccogliere....46
12.4 Scheda di osservazione....47
12.5 Questionario finale....47
12.6 Risultati e conclusioni da integrare....48
Riferimenti....48

1 Glossario

Termine	Significato nel progetto
Issue	Segnalazione appartenente a un progetto: bug, domanda, problema di documentazione o richiesta di funzionalità.
Reporter	Utente che ha creato la issue. La sua identità viene ricavata dal token verificato dal backend.
Assegnatario	Utente incaricato della issue. Può essere assente; se presente deve essere un membro del progetto.
Progetto	Contesto che raggruppa issue e membri e delimita l’accesso degli utenti normali.
Membership	Appartenenza tra un utente e un progetto; uno stesso utente può appartenere a più progetti.
USER e ADMIN	Ruoli globali dell’account. ADMIN gestisce utenti e progetti; il cambio stato resta riservato all’assegnatario.
SPA	Applicazione che aggiorna le schermate nel browser con React e gestisce la navigazione con React Router.
API HTTP	Percorsi del backend chiamati dal frontend per leggere o modificare i dati.
Middleware	Funzione eseguita nel percorso di una richiesta prima del controller oppure per gestire gli errori.
ORM	Strumento che collega oggetti JavaScript e tabelle PostgreSQL. Nel progetto viene usato Sequelize.
JWT	Token firmato inviato nell’header Authorization. Permette al backend di riconoscere l’utente.
Transazione	Insieme di salvataggi confermati tutti insieme oppure annullati in caso di errore.
Allegato	Singola immagine facoltativa, memorizzata nel database e scaricata tramite un endpoint protetto.
Notifica	Messaggio interno personale di attività o risoluzione, con indicazione letto/non letto.

2 Introduzione

BugBoard26 è un'applicazione web per la gestione collaborativa delle issue di progetti software. Il sistema permette di segnalare un problema, discuterlo attraverso commenti, assegnarlo a un membro del gruppo e seguirne l'avanzamento fino alla chiusura. La relazione descrive le funzionalità realizzate e le scelte tecniche necessarie a comprenderle e verificarle.

Il progetto è sviluppato per l'attività universitaria richiesta da SoftEngUniNA. La soluzione comprende un frontend React separato dal backend Node.js ed Express, con Sequelize e PostgreSQL per la persistenza. Le funzionalità sono disponibili attraverso API HTTP; i controlli decisivi vengono eseguiti nel backend.

2.1 Ambito del sistema

- Accesso tramite email e password e creazione degli account da parte di un amministratore.
- Gestione dei progetti e delle appartenenze come contesto delle segnalazioni.
- Creazione di issue con titolo, descrizione e tipo, priorità e immagine facoltative.
- Consultazione, filtri combinabili, assegnazione, commenti e avanzamento dello stato.
- Notifiche personali, in particolare al reporter quando una issue viene risolta.

2.2 Decisioni di ambito

Il riferimento della traccia a tutti gli utenti è applicato agli utenti autorizzati al progetto: gli USER operano nei progetti di cui sono membri; gli ADMIN mantengono l'accesso globale già previsto dal sistema. L'assegnazione può essere gestita da chi ha accesso alla issue, mentre il cambio di stato è consentito soltanto all'assegnatario, senza deroga per il ruolo ADMIN.

Non sono compresi registrazione autonoma, recupero password, email di notifica, modifica del profilo, ricerca testuale globale o gestione avanzata dei progetti. Alcuni relativi controlli sono presenti nell'interfaccia come elementi dimostrativi e non devono essere interpretati come funzionalità completate.

2.3 Organizzazione della relazione

La prima parte presenta utenti target e requisiti. La seconda descrive architettura, dati e interfaccia. La terza illustra build, avvio e versionamento. L'ultima documenta le verifiche automatiche e predispone le attività di valutazione dell'usabilità ancora da svolgere.

3 Raccolta e analisi dei requisiti

L’analisi parte dai punti funzionali della traccia e dalle regole di accesso concordate per il progetto. Le personas sono profili rappresentativi utili alle scelte di interfaccia; non descrivono partecipanti già coinvolti in un esperimento.

3.1 Utenti target e personas

Profilo	Obiettivo e contesto	Esigenze principali
Sviluppatrice del gruppo	Lavora quotidianamente alle issue assegnate e consulta commenti e segnalazioni.	Capire quale lavoro è da svolgere, leggere il contesto e aggiornare lo stato senza passaggi superflui.
Studente che segnala un problema	Utilizza il prodotto del progetto e descrive un malfunzionamento con una schermata.	Individuare il progetto, inviare una descrizione comprensibile e ricevere conferma della risoluzione.
Amministratore del gruppo	Prepara gli account, crea i progetti e ne organizza le appartenenze.	Distinguere il ruolo globale dall’appartenenza al singolo progetto e impedire accessi impropri.

3.2 Attori e responsabilità

L’utente non autenticato può raggiungere il login. L’utente autenticato autorizzato può leggere e creare issue e commenti, assegnare le issue e consultare le proprie notifiche. L’assegnatario aggiunge la facoltà di avanzare lo stato. L’amministratore può creare account USER o ADMIN, creare progetti e aggiungere o rimuovere membri.

Essere reporter, assegnatario e amministratore sono condizioni differenti: il reporter è fissato alla creazione, l’assegnazione può cambiare, il ruolo appartiene all’account. Il sistema mantiene queste informazioni separate per applicare correttamente i controlli.

3.3 Scenari principali

Uno studente segnala un bug allegando una schermata e lo assegna a una collega del progetto. La collega vede la notifica, legge il dettaglio e aggiunge un commento. Dopo aver preso in carico il lavoro, porta la issue da TODO a IN_PROGRESS e poi a CLOSED. Alla chiusura il reporter riceve una notifica di risoluzione. In parallelo un amministratore può mantenere aggiornati gli account e i membri senza ottenere automaticamente il permesso di cambiare lo stato delle issue altrui.

4 Specifica dei requisiti

4.1 Diagramma dei casi di uso

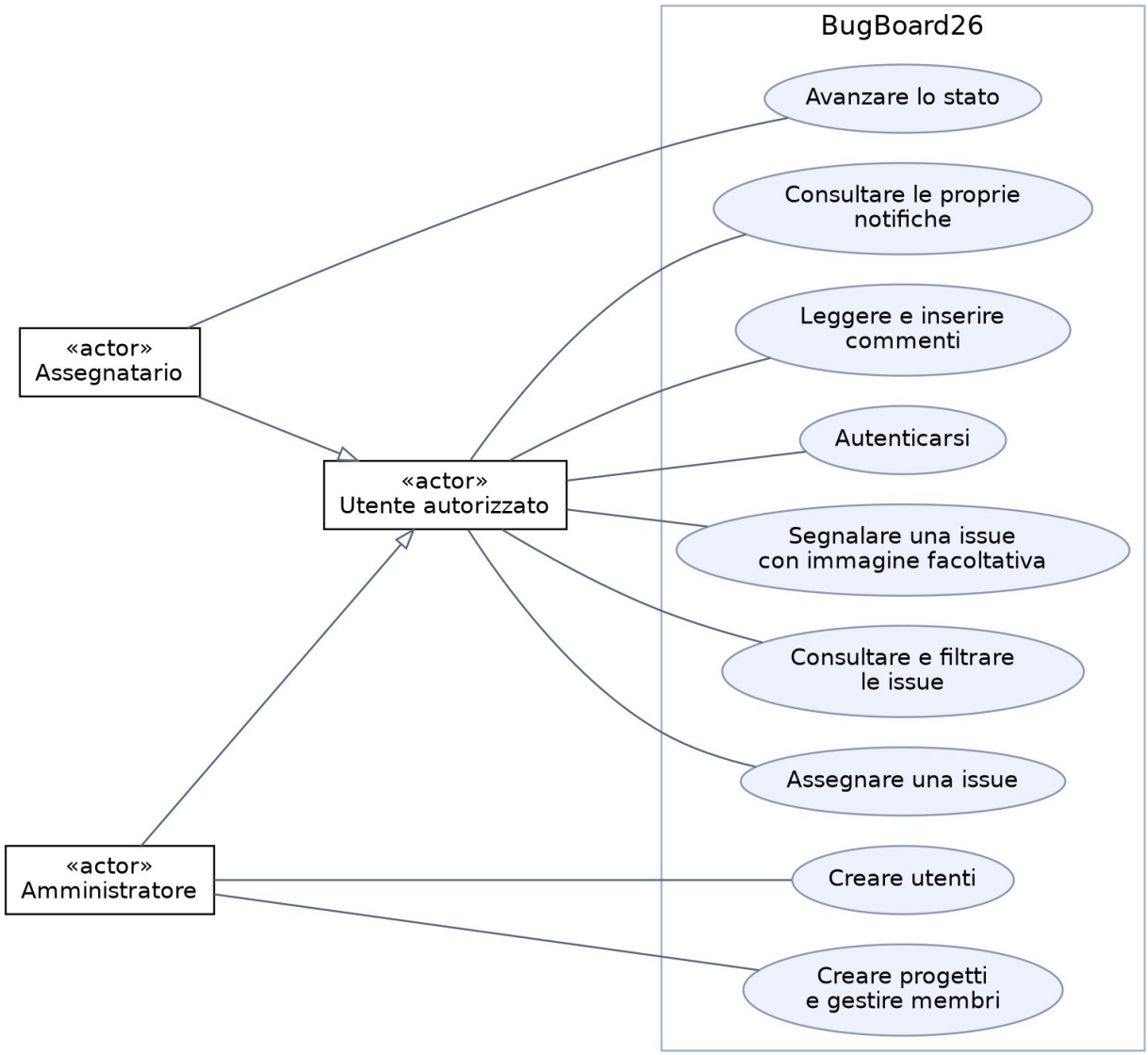


Figura 1 Attori e casi di uso principali di BugBoard26

Le associazioni con l'attore Utente autorizzato presuppongono l'accesso al progetto. I casi di amministrazione sono riservati ad ADMIN. La specializzazione Assegnatario rappresenta una relazione con una specifica issue e non un ulteriore ruolo globale.

4.2 Requisiti funzionali

ID	Funzionalità e comportamento	Origine
RF01	Autenticazione. Il sistema autentica tramite email e password; credenziali errate non permettono l’accesso.	Traccia 1
RF02	Amministrazione account. Un ADMIN crea utenze USER o ADMIN con email unica e password protetta; è disponibile un admin iniziale di sviluppo.	Traccia 1
RF03	Contesto di progetto. Un ADMIN crea progetti e gestisce membri; un USER vede i propri progetti.	Contesto e regola di accesso
RF04	Creazione issue. Un utente autorizzato indica titolo, descrizione e tipo; il server imposta reporter e stato TODO.	Traccia 2
RF05	Priorità facoltativa. È possibile omettere la priorità o scegliere LOW, MEDIUM, HIGH o CRITICAL.	Traccia 2
RF06	Immagine facoltativa. È possibile allegare una PNG, JPEG o WebP entro 5 MB e 16 megapixel; il file resta privato.	Traccia 2, possibilità gradita
RF07	Riepilogo e filtri. La dashboard mostra le issue e permette di combinare tipo, stato e priorità, incluse le priorità non specificate.	Traccia 3
RF08	Assegnazione. L’issue può essere assegnata a un membro del progetto, riassegnata o lasciata senza assegnatario.	Contesto e traccia 6
RF09	Commenti. Ogni utente autorizzato al progetto può leggere e aggiungere messaggi alla issue.	Traccia 5
RF10	Avanzamento stato. Solo l’assegnatario può avanzare TODO a IN_PROGRESS e IN_PROGRESS a CLOSED.	Traccia 6 e regola concordata
RF11	Notifiche personali. Il sistema notifica assegnazioni, commenti e attività ai destinatari previsti; la chiusura notifica sempre il reporter.	Traccia 6
RF12	Lettura notifiche. Ogni utente consulta e marca lette soltanto le proprie notifiche.	Supporto a RF11

4.3 Requisiti non funzionali

ID	Requisito	Verifica prevista o svolta
RNF01	Proteggere identità, password e accessi ai dati.	JWT verificato, hash BCrypt, controlli sulle API e test di accesso negato. HTTPS va predisposto nel deployment esterno.
RNF02	Mantenere coerenti i salvataggi composti.	Transazioni per issue, immagine e notifiche; test di rollback e richieste concorrenti.
RNF03	Conservare i dati dopo il riavvio.	PostgreSQL persistente e test con riavvio effettivo del processo server.
RNF04	Fornire feedback comprensibili e prevenire invii non validi.	Controlli sui form, messaggi di errore e stati di caricamento; valutazione con utenti da completare.
RNF05	Consentire manutenzione e avvio ripetibile.	Moduli con responsabilità riconoscibili, lockfile npm, script SQL e Dockerfile backend.
RNF06	Fornire una navigazione rapida e una risposta adeguata al carico previsto.	Filtri locali e richieste asincrone; non sono ancora disponibili misure di carico o tempi massimi garantiti.

4.4 Requisiti di dominio

RD01. Ogni issue appartiene a un solo progetto e ha un solo reporter. Il reporter deriva dall'identità autenticata e non è scelto dal client.

RD02. I tipi sono QUESTION, BUG, DOCUMENTATION e FEATURE. L'eventuale priorità appartiene a LOW, MEDIUM, HIGH e CRITICAL.

RD03. La priorità assente è memorizzata come NULL. L'assegnatario può essere NULL; l'immagine è una relazione facoltativa uno a uno.

RD04. L'assegnatario deve essere membro del progetto al momento dell'assegnazione, anche se ha ruolo ADMIN.

RD05. Lo stato iniziale è TODO. Sono ammessi soltanto gli avanzamenti TODO a IN_PROGRESS e IN_PROGRESS a CLOSED. Ripetere lo stato corrente da parte dell'assegnatario non crea nuovi eventi.

RD06. Solo l'assegnatario corrente può richiedere il cambio di stato. Una riassegnazione trasferisce subito tale facoltà; senza assegnatario non è possibile avanzare.

RD07. Un utente normale rimosso dal progetto perde l'accesso a issue, commenti e immagini. Le attribuzioni storiche non vengono cancellate.

RD08. La risoluzione notifica il reporter anche se chiude la propria issue o non è più membro. In quest'ultimo caso la notifica non espone un collegamento al dettaglio non più accessibile.

RD09. La notifica di assegnazione va al nuovo assegnatario; le notifiche ordinarie escludono chi ha compiuto l'azione e non duplicano il destinatario.

RD10. I commenti sono ordinati dal più vecchio al più recente. Anche dopo la chiusura è possibile commentare e cambiare assegnatario, ma lo stato non si riapre.



Figura 2 Macchina degli stati della issue e vincoli principali

4.5 Casi di uso nel modello di Cockburn

Le schede seguenti descrivono obiettivo, attori, precondizioni, scenario principale ed estensioni. Le condizioni di fallimento indicano gli effetti osservabili; i dati già persistenti restano conservati quando un'operazione viene rifiutata o annullata.

UC01 Autenticarsi

Campo	Descrizione
Goal in context	Accedere alle funzioni associate al proprio account.
Attore primario	Utente con account esistente
Precondizioni	Il backend e il database sono disponibili.
Trigger	Invio del form Login.
Condizione finale di successo	L'utente raggiunge la pagina progetti con token e dati pubblici dell'account.
Condizione finale di fallimento	Nessuna sessione autenticata viene creata.

Passo	Attore	Sistema
1	Inserisce email e password.	Controlla formato e presenza dei dati.
2	Invia il form.	Cerca l'account, confronta BCrypt e firma il JWT.
3	Accede ai propri progetti.	Restituisce i dati e aggiorna la navigazione.

Estensioni e alternative. Credenziali non valide: messaggio uniforme. Token assente o scaduto nelle richieste successive: 401 e ritorno al login.

UC02 Creare un account

Campo	Descrizione
Goal in context	Fornire a un collaboratore un’utenza USER oppure ADMIN.
Attore primario	Amministratore
Precondizioni	L’amministratore è autenticato.
Trigger	Invio del form di creazione utente.
Condizione finale di successo	L’account è persistente con email normalizzata e password in forma di hash.
Condizione finale di fallimento	Non viene creato un account parziale.

Passo	Attore	Sistema
1	Indica nome, email, password e ruolo.	Valida i campi e verifica l’unicità dell’email.
2	Conferma la creazione.	Calcola l’hash BCrypt e salva l’account.
3	Legge il risultato.	Mostra l’utente senza password né hash.

Estensioni e alternative. Email già usata: 409. Dati non validi: 400. Utente non ADMIN: 403. La creazione account e l’aggiunta a un progetto sono operazioni distinte.

UC03 Creare un progetto e gestirne i membri

Campo	Descrizione
Goal in context	Predisporre il contesto di lavoro e gli utenti che possono parteciparvi.
Attore primario	Amministratore
Precondizioni	Esiste una sessione ADMIN e gli account da aggiungere.
Trigger	Creazione progetto oppure azione sui membri.
Condizione finale di successo	Il progetto e le appartenenze sono memorizzati. Il creatore è aggiunto come membro.
Condizione finale di fallimento	Una creazione fallita non lascia un progetto senza appartenenza del creatore.

Passo	Attore	Sistema
1	Indica nome e descrizione del progetto.	Salva progetto e appartenenza in transazione.
2	Sceglie un account esistente.	Aggiunge l’appartenenza se non duplicata.
3	Rimuove un membro quando necessario.	Revoca l’accesso degli USER nelle richieste successive.

Estensioni e alternative. Dati mancanti: 400. Account non esistente: 404. Appartenenza duplicata: 409. Le issue conservano le attribuzioni storiche.

UC04 Segnalare una issue con immagine facoltativa

Campo	Descrizione
Goal in context	Descrivere un problema o una richiesta nel progetto.
Attore primario	Utente autorizzato al progetto
Precondizioni	L’utente è autenticato e può accedere al progetto.
Trigger	Apertura e invio di Create Issue.
Condizione finale di successo	Issue in TODO con reporter corretto; eventuale immagine e notifica salvate insieme.
Condizione finale di fallimento	Nessuna issue, immagine o notifica parziale resta salvata.

Passo	Attore	Sistema
1	Compila titolo, descrizione e tipo.	Mantiene facoltative priorità, assegnazione e immagine.
2	Sceglie eventualmente un file e un membro.	Controlla accesso, appartenenza, formato e dimensioni reali del file.
3	Invia il form.	Salva issue, immagine e notifica in transazione.
4	Legge il dettaglio.	Restituisce 201 e carica l’immagine con una richiesta autenticata.

Estensioni e alternative. File falso, corrotto o non supportato: 400. File oltre 5 MB: 413. Errore di salvataggio: rollback. Annulla chiude il modulo senza creare la issue.

UC05 Consultare e filtrare le issue

Campo	Descrizione
Goal in context	Trovare le segnalazioni pertinenti attraverso il riepilogo del progetto.
Attore primario	Utente autorizzato al progetto
Precondizioni	Il progetto è accessibile.
Trigger	Apertura della dashboard o modifica di un filtro.
Condizione finale di successo	La dashboard mostra soltanto le issue corrispondenti ai filtri.
Condizione finale di fallimento	Un errore di caricamento è visibile e non viene presentato come lista vuota valida.

Passo	Attore	Sistema
1	Apre la dashboard di un progetto.	Carica issue e dati riepilogativi tramite API.
2	Seleziona tipo, stato o priorità.	Combina i filtri nel browser e aggiorna il riepilogo.
3	Apre una issue o azzera i filtri.	Mostra il dettaglio oppure l’elenco completo del progetto.

Estensioni e alternative. Nessuna corrispondenza: messaggio dedicato e azzeramento dei filtri. Priorità non specificata: filtro UNSPECIFIED. Accesso non consentito: 403.

UC06 Assegnare o riassegnare una issue

Campo	Descrizione
Goal in context	Indicare la persona incaricata della lavorazione.
Attore primario	Utente autorizzato al progetto
Precondizioni	La issue esiste e il destinatario è un membro corrente.
Trigger	Selezione in Change Assignee oppure scelta durante la creazione.
Condizione finale di successo	L’assegnatario è aggiornato e riceve la notifica prevista.
Condizione finale di fallimento	L’assegnazione precedente resta invariata.

Passo	Attore	Sistema
1	Sceglie un membro oppure Unassigned.	Controlla l’identità del richiedente e l’appartenenza del destinatario.
2	Conferma attraverso la selezione.	Aggiorna issue e progetto e salva l’eventuale notifica in transazione.
3	Legge il nuovo assegnatario.	Restituisce il dettaglio aggiornato e adegua il controllo dello stato.

Estensioni e alternative. Destinatario esterno al progetto: 400. Stesso assegnatario: nessun evento duplicato. Autoassegnazione: nessuna notifica ordinaria a se stessi.

UC07 Commentare una issue

Campo	Descrizione
Goal in context	Condividere un aggiornamento o una richiesta di chiarimento.
Attore primario	Utente autorizzato al progetto
Precondizioni	La issue è accessibile, anche se già chiusa.
Trigger	Invio di Add Comment.
Condizione finale di successo	Il commento compare con autore autenticato e data; le notifiche sono coerenti.
Condizione finale di fallimento	Il testo inserito resta disponibile nel form dopo un errore.

Passo	Attore	Sistema
1	Scrive un messaggio non vuoto.	Controlla lunghezza e accesso alla issue.
2	Invia il commento.	Salva autore, contenuto e notifiche in una transazione.
3	Legge il risultato nella discussione.	Inserisce il nuovo commento nell’ordine cronologico previsto.

Estensioni e alternative. Testo vuoto o oltre 5000 caratteri: 400. Appartenenza revocata: 403. Errore nel salvataggio delle notifiche: annullamento dell’operazione.

UC08 Avanzare lo stato e notificare la risoluzione

Campo	Descrizione
Goal in context	Registrare la presa in carico e la conclusione della lavorazione.
Attore primario	Assegnatario corrente
Precondizioni	L’utente ha accesso alla issue ed è il suo assegnatario.
Trigger	Selezione del successivo stato in Change Status.
Condizione finale di successo	Stato aggiornato e, alla chiusura, una notifica di risoluzione per il reporter.
Condizione finale di fallimento	Stato, data del progetto e notifiche rimangono coerenti con la situazione precedente.

Passo	Attore	Sistema
1	Sceglie IN_PROGRESS da TODO.	Rilegge e blocca la issue, poi verifica assegnatario e transizione.
2	Dopo la risoluzione sceglie CLOSED.	Aggiorna lo stato e crea la notifica al reporter nella stessa transazione.
3	Legge la conferma.	Restituisce il dettaglio aggiornato senza duplicare eventi su richieste ripetute.

Estensioni e alternative. Non assegnatario, compreso ADMIN: 403. Salto o ritorno di stato: 409. Issue senza assegnatario: 403. Scrittura della notifica fallita: rollback.

UC09 Consultare e leggere le notifiche

Campo	Descrizione
Goal in context	Conoscere assegnazioni, aggiornamenti e risoluzioni che riguardano l’utente.
Attore primario	Utente autenticato
Precondizioni	Esiste una sessione valida.
Trigger	Apertura del pannello Notifications.
Condizione finale di successo	Sono mostrate soltanto notifiche personali e il conteggio non lette è aggiornato.
Condizione finale di fallimento	Non è possibile leggere o marcare notifiche appartenenti ad altri account.

Passo	Attore	Sistema
1	Apre il pannello.	Carica le notifiche personali e il numero di non lette.
2	Seleziona una notifica.	La marca letta e apre la issue soltanto se è ancora accessibile.
3	Consulta una risoluzione di un progetto lasciato.	Mostra il messaggio senza collegamento al dettaglio riservato.

Estensioni e alternative. Notifica altrui o inesistente: 404. Rilettura della stessa notifica: effetto idempotente. Errore di rete: messaggio con possibilità di riprovare.

5 Progettazione architetturale e scelte tecnologiche

5.1 Descrizione della architettura

Il sistema separa presentazione, logica applicativa e persistenza. Il frontend è una SPA React; il backend espone API con Express; PostgreSQL conserva gli account e i dati collaborativi. Questa suddivisione permette di seguire una richiesta senza introdurre livelli che non servono al progetto.

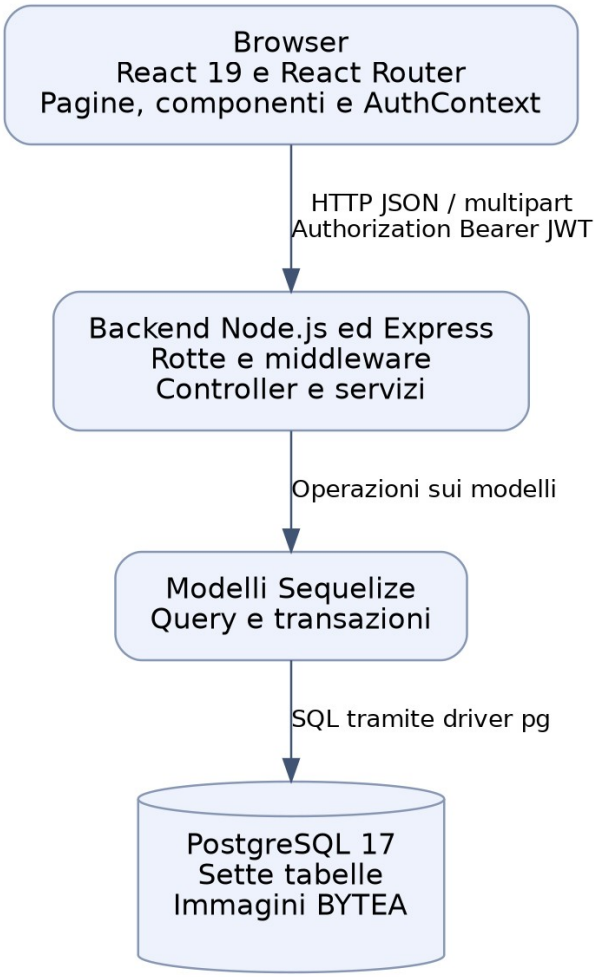


Figura 3 Componenti principali e comunicazione tra frontend e backend

Nel backend il percorso è richiesta, rotta, middleware, controller, eventuale servizio, modello Sequelize e risposta. Le operazioni semplici possono usare direttamente i modelli dal controller; le regole condivise e i salvataggi composti stanno nei servizi. Non esiste un livello repository separato e non vengono definite classi DTO: i JSON pubblici sono preparati da funzioni esplicite.

5.2 Criteri di design e motivazioni

La scelta di JavaScript con import/export rende uniforme il linguaggio tra i due lati dell'app e mantiene una struttura riconoscibile per lo studio. Express associa i percorsi alle funzioni senza dipendere da annotazioni. Sequelize mantiene il modello a oggetti e centralizza la configurazione del database. Le classi che estendono Model descrivono le entità, mentre create, update e destroy eseguono i salvataggi.

La migrazione da Spring Boot ha conservato PostgreSQL, gli identificativi, le password BCrypt e l'autenticazione JWT. Le API esistenti sono rimaste utilizzabili dal frontend. Le successive modifiche funzionali hanno reso facoltativa la priorità, sostituito OPEN con TODO e riservato il cambio stato all'assegnatario.

Per gli allegati si usa una tabella separata con i byte dell'immagine. In un progetto di queste dimensioni ciò consente di salvare issue e allegato nella stessa transazione e di conservare tutto con il database. Le liste caricano soltanto i metadati: i byte sono letti quando viene richiesto l'endpoint dell'immagine. Questa scelta privilegia la semplicità e non presuppone uno storage cloud.

5.3 Tecnologie adottate

Componente	Tecnologia	Responsabilità
Interfaccia	React 19, React Router	Componenti, stato locale, navigazione e protezione delle pagine.
Build frontend	Vite 8	Server di sviluppo e compilazione degli asset statici.
Client HTTP	Axios	Richieste alle API, Bearer token e gestione condivisa degli errori 401.
Esecuzione backend	Node.js 22 o successivo, JavaScript ESM	Esecuzione dei moduli; immagine Docker basata su Node 24 Alpine.
HTTP e middleware	Express 5, cors, multer	Routing, policy del browser e ricezione controllata del file multipart.
Dati	Sequelize 6, driver pg, PostgreSQL 17	Modelli, relazioni, query, transazioni e vincoli persistenti.
Autenticazione	bcryptjs e jsonwebtoken	Hash delle password e firma/verifica dei token HS256.
Immagini	Sharp	Decodifica, verifica di formato e dimensioni, normalizzazione e rimozione dei metadati.
Verifica	node:test, Vitest, Testing Library, jsdom	Test delle API reali e dei componenti del frontend.

Le versioni riproducibili delle dipendenze sono contenute nei package-lock.json. Le spiegazioni delle librerie sono riferite alle documentazioni ufficiali [R1-R6]; le regole specifiche di BugBoard26 derivano dal codice del progetto.

5.4 Persistenza dei dati

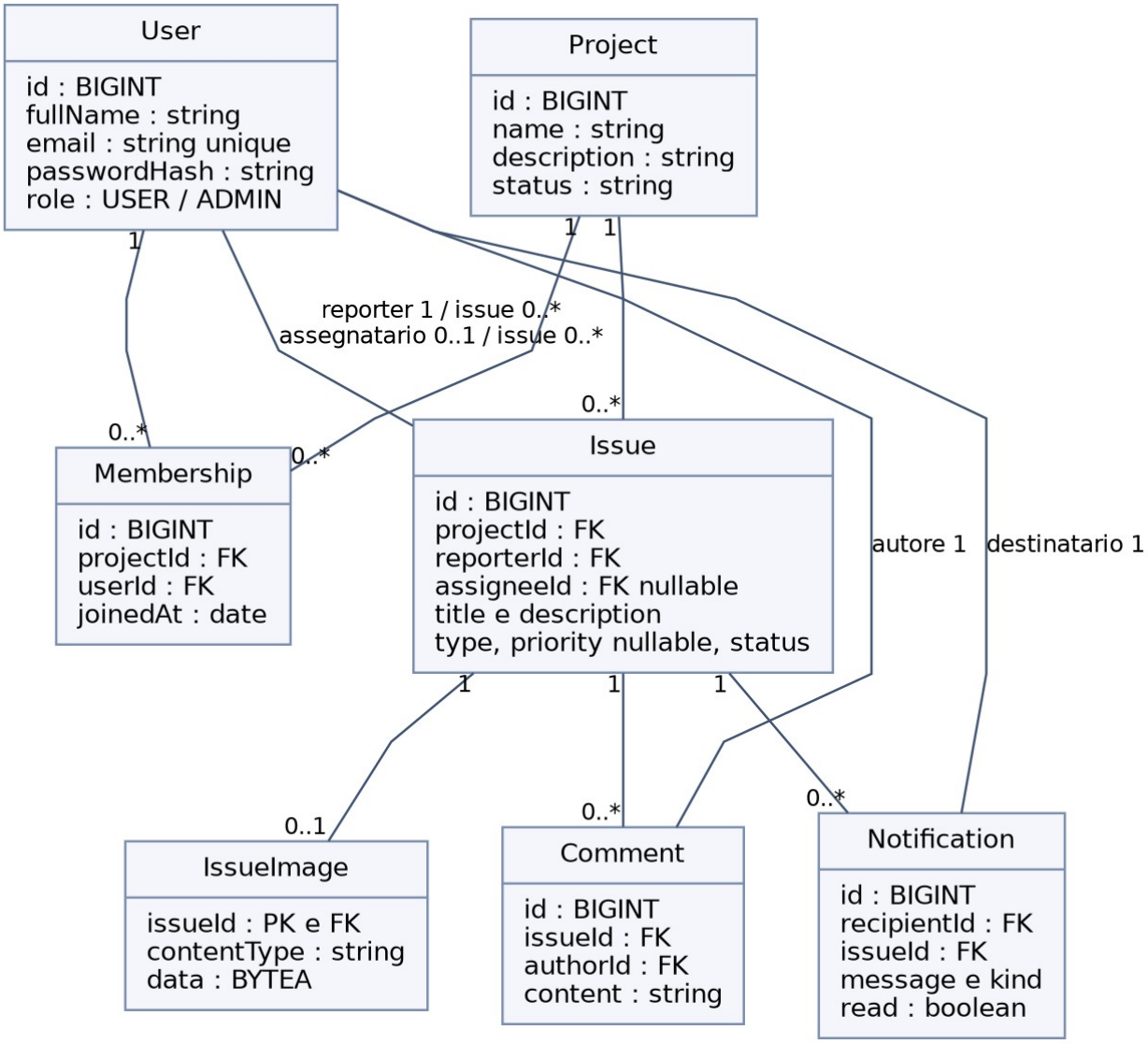


Figura 4 Modello delle sette entità persistenti con cardinalità principali

User e Project sono collegati attraverso Membership, che impone l'unicità della coppia progetto e utente. Issue collega progetto, reporter e assegnatario facoltativo. Comment associa un messaggio alla issue e al suo autore. Notification collega una issue a un destinatario. IssueImage usa issue_id come chiave primaria e chiave esterna, consentendo al massimo un'immagine per issue.

Tabella	Dati e vincoli rilevanti
app_users	Email unica, hash della password, ruolo USER/ADMIN e dati pubblici del profilo.
projects	Nome, descrizione, stato e date. L'interfaccia non espone una gestione completa dell'archiviazione.
project_memberships	Chiavi esterne a utente e progetto, data di ingresso e coppia unica.
issues	Titolo fino a 200 caratteri, descrizione fino a 10000, tipo, priorità nullable, stato e riferimenti alle persone.
issue_comments	Contenuto fino a 5000 caratteri, autore e data.

Tabella	Dati e vincoli rilevanti
notifications	Destinatario, messaggio, kind ACTIVITY/RESOLUTION, indicazione di lettura e data.
issue_images	Tipo PNG/JPEG/WebP e dati BYTEA da 1 a 5242880 byte; la chiave esterna evita allegati senza issue.

5.5 Inizializzazione e conservazione dei dati

Il server controlla la connessione e la presenza delle tabelle senza usare sync con force o alter. Il comando db:init esegue esplicitamente gli script 001-initial.sql, 002-issue-requirements.sql e 003-issue-images.sql. Il secondo conserva i dati e converte OPEN in TODO; il terzo aggiunge soltanto la tabella degli allegati. L’inizializzazione ripetuta è verificata dai test.

5.6 Autenticazione sessione e autorizzazione

Il login confronta la password inviata con l'hash BCrypt. In caso di successo genera un JWT HS256 con identificativo utente, issuer, audience e scadenza. Il frontend memorizza token e dati pubblici della sessione nel localStorage e invia Authorization: Bearer nelle richieste protette. Non esiste una sessione Express conservata sul server.

Il middleware verifica il token e rilegge l'account dal database a ogni richiesta. Il ruolo quindi non è accettato dal browser. I servizi verificano inoltre l'accesso al progetto e, quando richiesto, l'identità dell'assegnatario. CORS limita le origini ammesse nel browser; non sostituisce né autenticazione né autorizzazione.

Alla riapertura dell'app, AuthContext verifica la sessione con GET /users/me. Un errore 401 riferito al token corrente elimina la sessione locale e riporta al login. Logout elimina il token dal browser; non lo revoca sul server. Non sono implementati refresh token, recupero password o un elenco di revoca dei JWT.

La configurazione locale opzionale di accesso automatico serve soltanto alle prove con Vite ed è esclusa dalla build di produzione. Le chiavi JWT e le credenziali reali restano nei file di ambiente locali. In un deployment esterno occorrono HTTPS e una chiave non dimostrativa.

5.7 Gestione delle immagini e delle transazioni

Quando è selezionato un file, il frontend invia FormData alla stessa POST /issues; senza file conserva il JSON. Multer riceve al massimo un'immagine da 5 MB. Sharp controlla il formato effettivo, rifiuta file falsi o corrotti, applica il limite di 16 megapixel, esclude le animazioni e riscrive l'immagine senza i metadati. Non viene usato il nome del file per costruire percorsi su disco.

La creazione della issue, l'eventuale immagine, l'aggiornamento del progetto e la notifica sono salvati in una transazione. Un errore annulla tutti i salvataggi. Le modifiche di stato bloccano la riga della issue durante la transazione: richieste identiche concorrenti non producono notifiche duplicate.

GET /issues/{id}/image controlla il token e l'accesso alla issue prima di restituire i byte. La risposta usa il tipo verificato, Cache-Control private no-store e X-Content-Type-Options nosniff. Il frontend scarica un Blob con Axios e crea un URL temporaneo, liberato alla chiusura del componente. Nessun token compare nell'URL dell'immagine.

6 Design della interfaccia utente

L’interfaccia distingue pagina di accesso, selezione del progetto, dashboard e dettaglio. Sidebar e barra superiore mantengono il contesto della navigazione; nel dettaglio si distinguono descrizione, immagine, discussione e dati di assegnazione. Le figure mostrano schermate acquisite dalla build effettiva con API e PostgreSQL, usando dati dimostrativi separati dai dati di sviluppo.

6.1 Colori e tipografia

Elemento	Scelta nel CSS	Scopo
Sfondo e pannelli	#f6f8fc e #ffffff	Separare le aree di lavoro con superfici chiare e bordi visibili.
Testo principale e secondario	#0f172a e #64748b	Distinguere contenuto, etichette e informazioni accessorie.
Azioni e navigazione	#2563eb, accento forte #1d4ed8	Rendere riconoscibili i controlli principali e il contesto selezionato.
Stati e priorità	Badge colorati accompagnati da testo	Fornire una distinzione visiva senza affidare il significato soltanto al colore.
Carattere	Inter, Segoe UI, sans-serif come fallback	Mantenere leggibilità nei controlli e nelle liste. Non è necessario un font remoto.

6.2 Accesso e scelta del progetto

Il login presenta email, password, messaggio di errore e azione di invio. Dopo l’accesso, Projects mostra i progetti visibili e i conteggi. Gli ADMIN dispongono anche del modulo di creazione. Le funzioni dimostrative di recupero password e Remember me non modificano il flusso di autenticazione implementato.

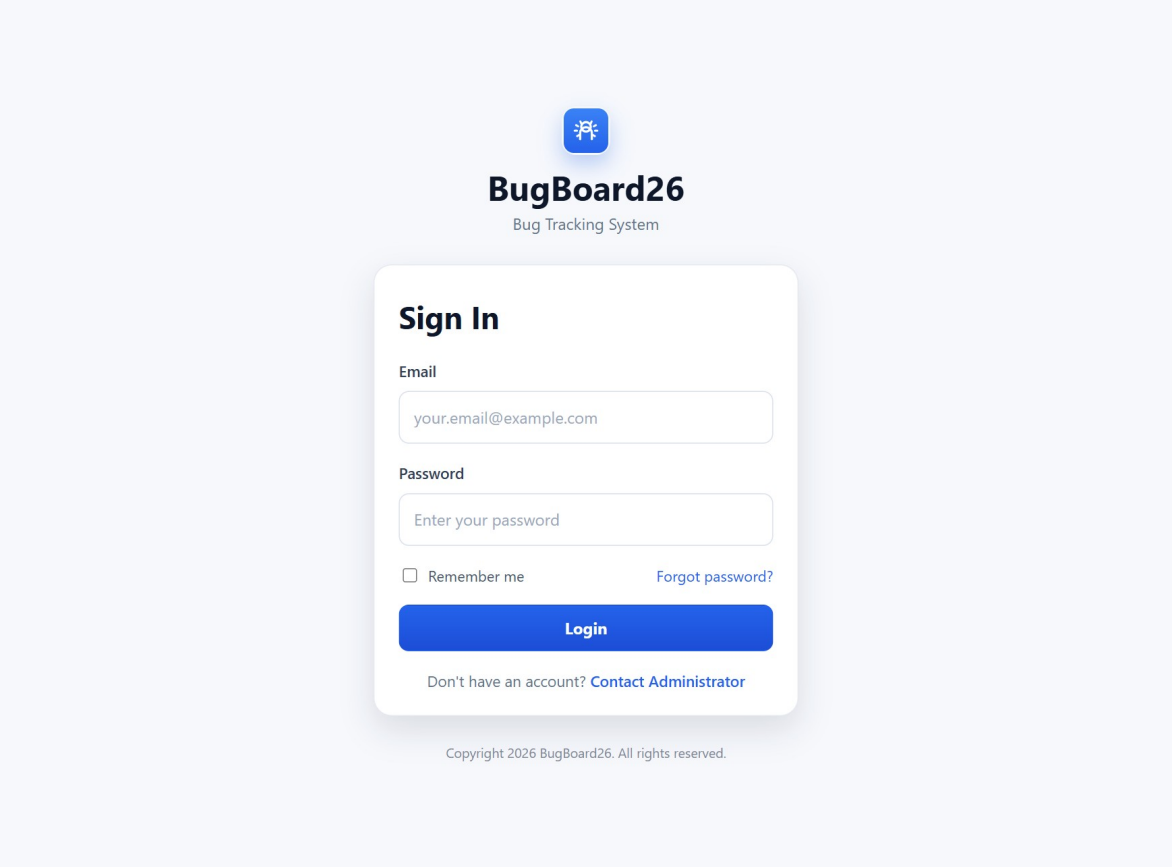


Figura 5 Pagina di accesso con i campi di autenticazione

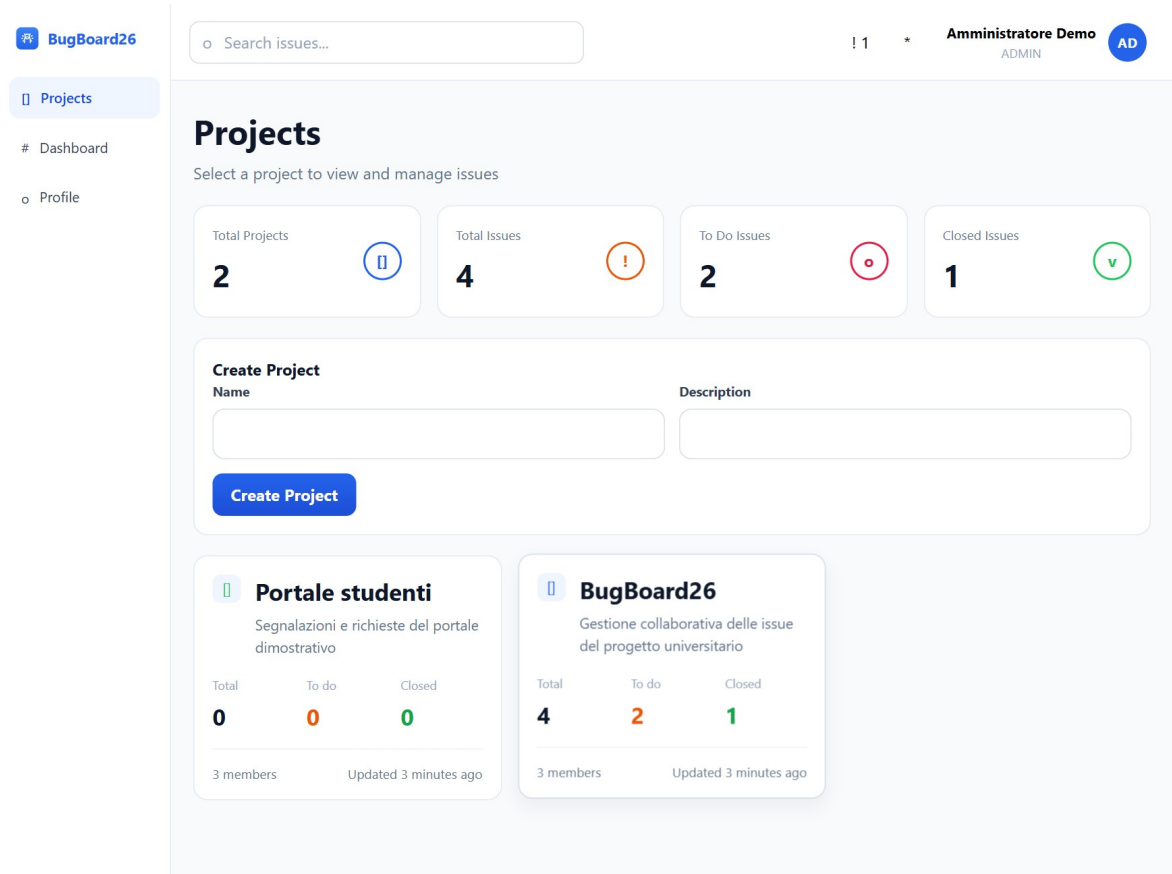


Figura 6 Elenco progetti e creazione disponibile per ADMIN

6.3 Dashboard e filtri

La dashboard presenta i filtri prima dell'elenco delle issue. Tipo, stato e priorità si combinano; Clear filters ripristina la vista del progetto. Una priorità omessa viene mostrata come non specificata e può essere selezionata come filtro. I filtri sono applicati nel browser ai dati già autorizzati restituiti dalle API.

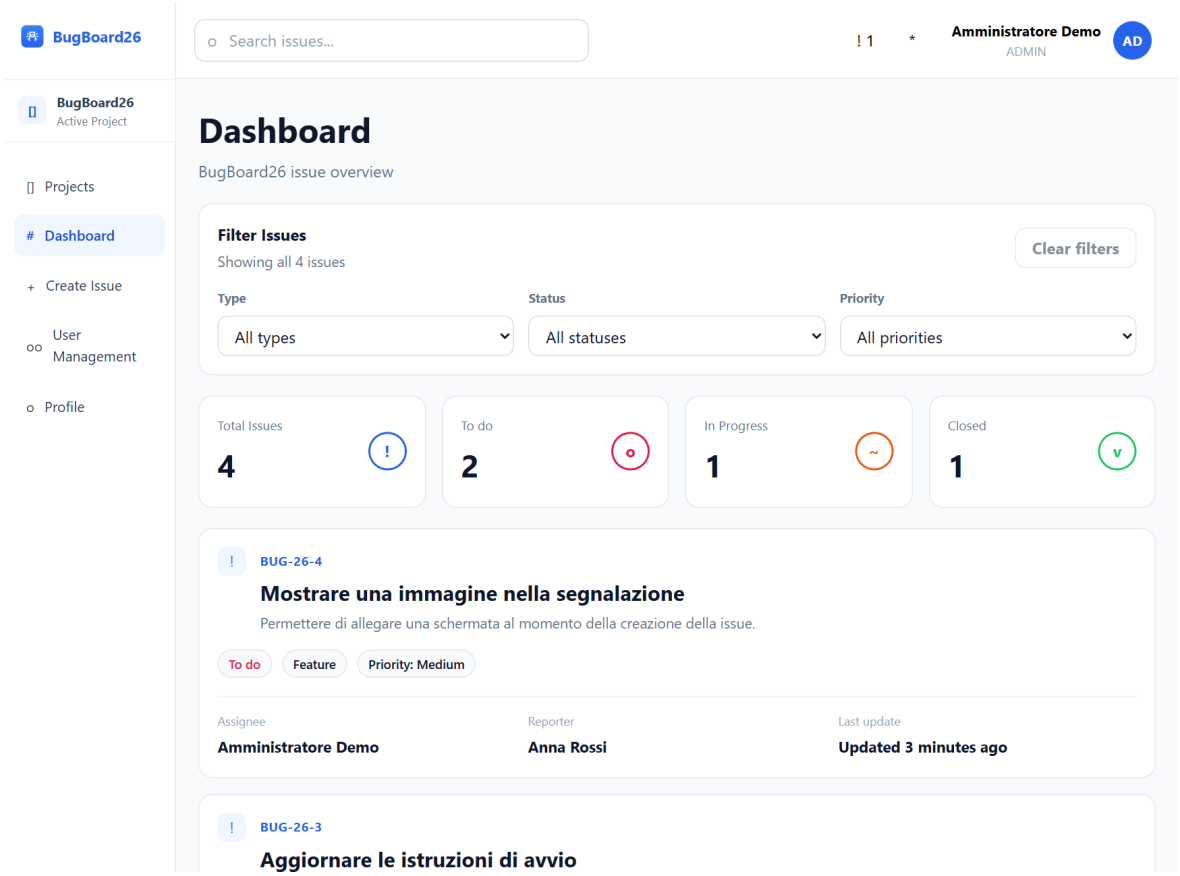


Figura 7 Dashboard del progetto con riepilogo e issue

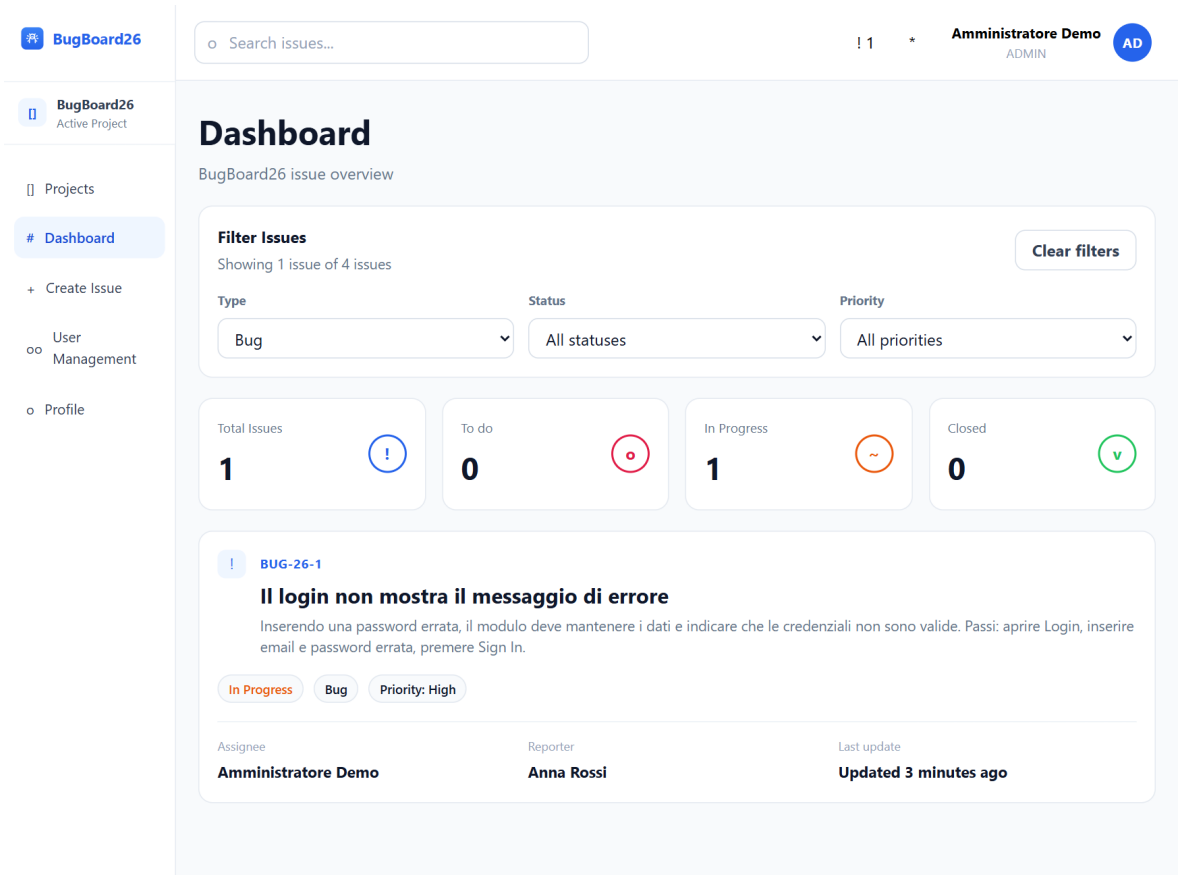


Figura 8 Filtro per il tipo BUG applicato alla dashboard

6.4 Creazione e prevenzione degli errori

Create Issue è un dialogo che mantiene il contesto del progetto. Titolo, descrizione e tipo sono obbligatori; priorità, assegnatario e immagine sono facoltativi. Il selettore dell'assegnatario usa i membri del progetto. La scelta di un file non valido mostra subito un errore; il backend ripete i controlli anche se l'interfaccia viene aggirata.

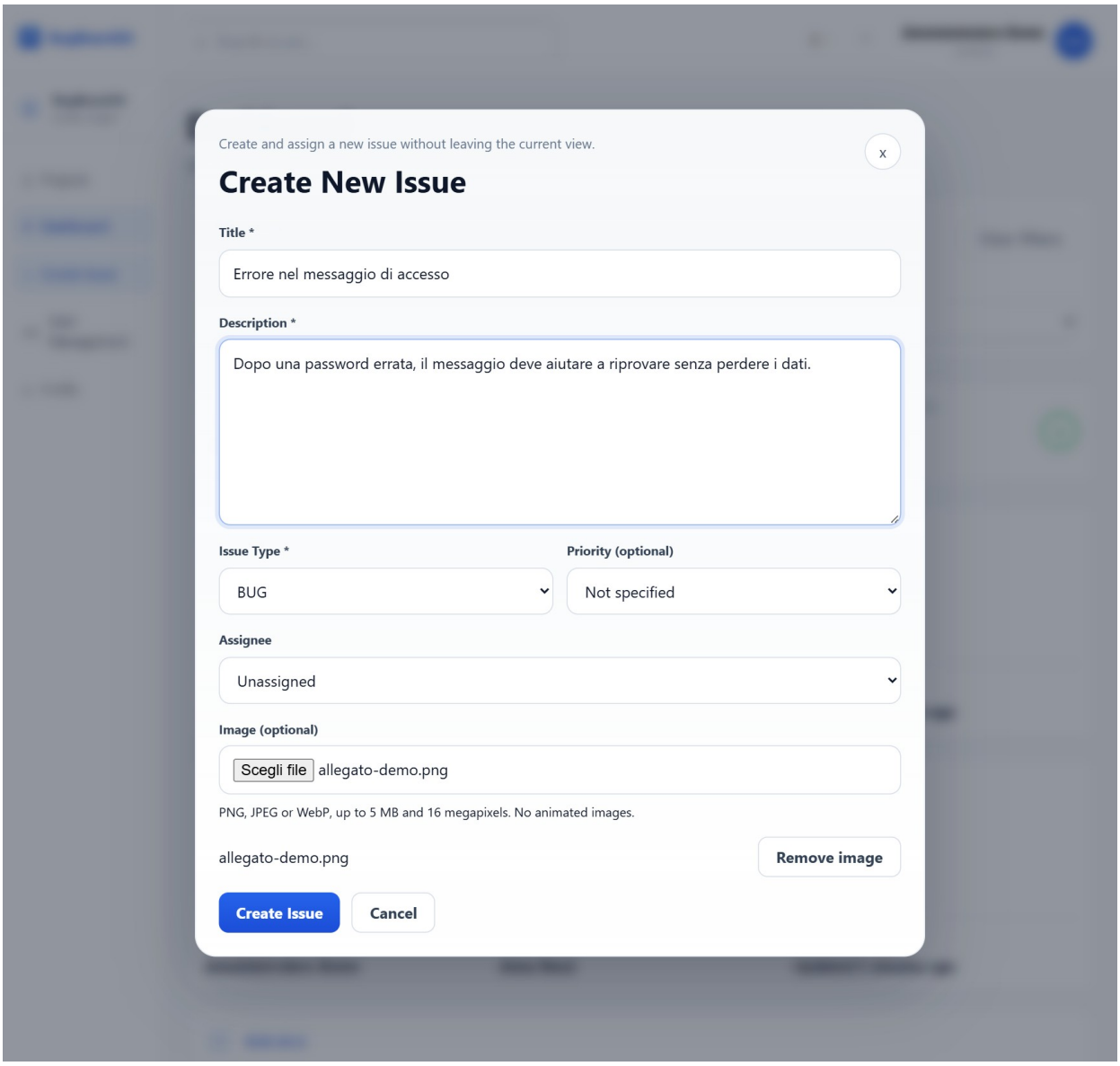


Figura 9 Creazione di una issue con immagine selezionata e priorità omessa

Durante l'invio il pulsante principale è disabilitato per limitare invii ripetuti. Se il backend rifiuta il file o la richiesta, i dati del form restano visibili. Remove image permette di rinunciare all'allegato prima dell'invio; Cancel chiude il dialogo senza effettuare il salvataggio.

6.5 Dettaglio commenti e assegnazione

Il dettaglio raggruppa titolo, tipo, priorità e stato nella parte superiore. La descrizione e l'immagine precedono la discussione, mentre il pannello laterale raccoglie assegnatario, reporter e date. Questa vicinanza rende riconoscibili le informazioni che appartengono alla stessa issue.

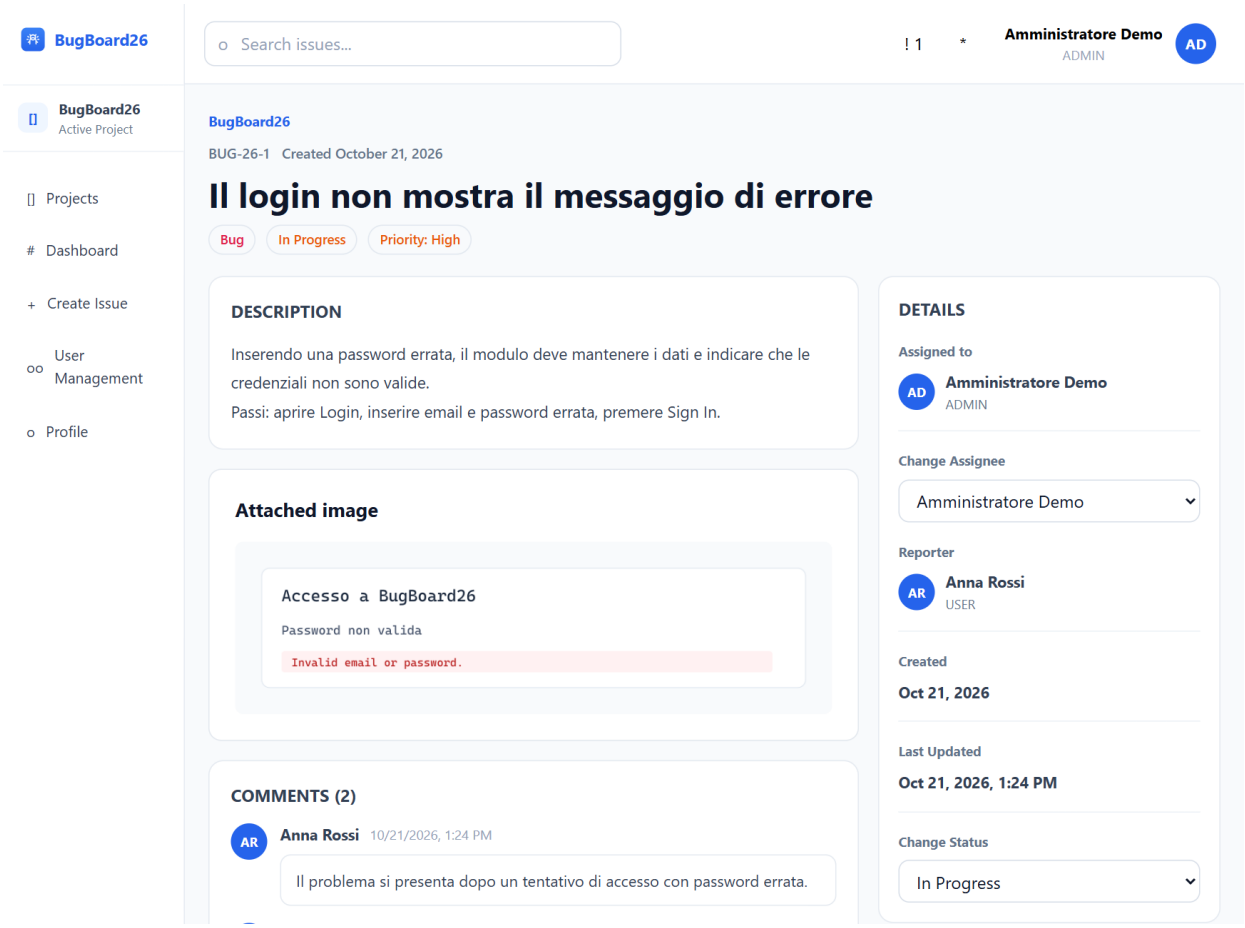


Figura 10 Dettaglio della issue con allegato e dati di assegnazione

Change Status è disabilitato per chi non è assegnatario e per una issue chiusa. Senza assegnatario compare l'indicazione di assegnare prima la issue. Il controllo è solo un aiuto visivo: il permesso viene sempre verificato dal backend. I commenti riportano autore e data; un invio fallito mantiene il testo scritto.

6.6 Feedback e notifiche

Gli stati di caricamento indicano che una richiesta è in corso; i messaggi di errore restano visibili finché l'utente può intervenire. L'immagine offre Retry se il download fallisce. Il pannello notifiche rende riconoscibili i messaggi non letti e viene aggiornato all'apertura, dopo le operazioni e periodicamente ogni 30 secondi.

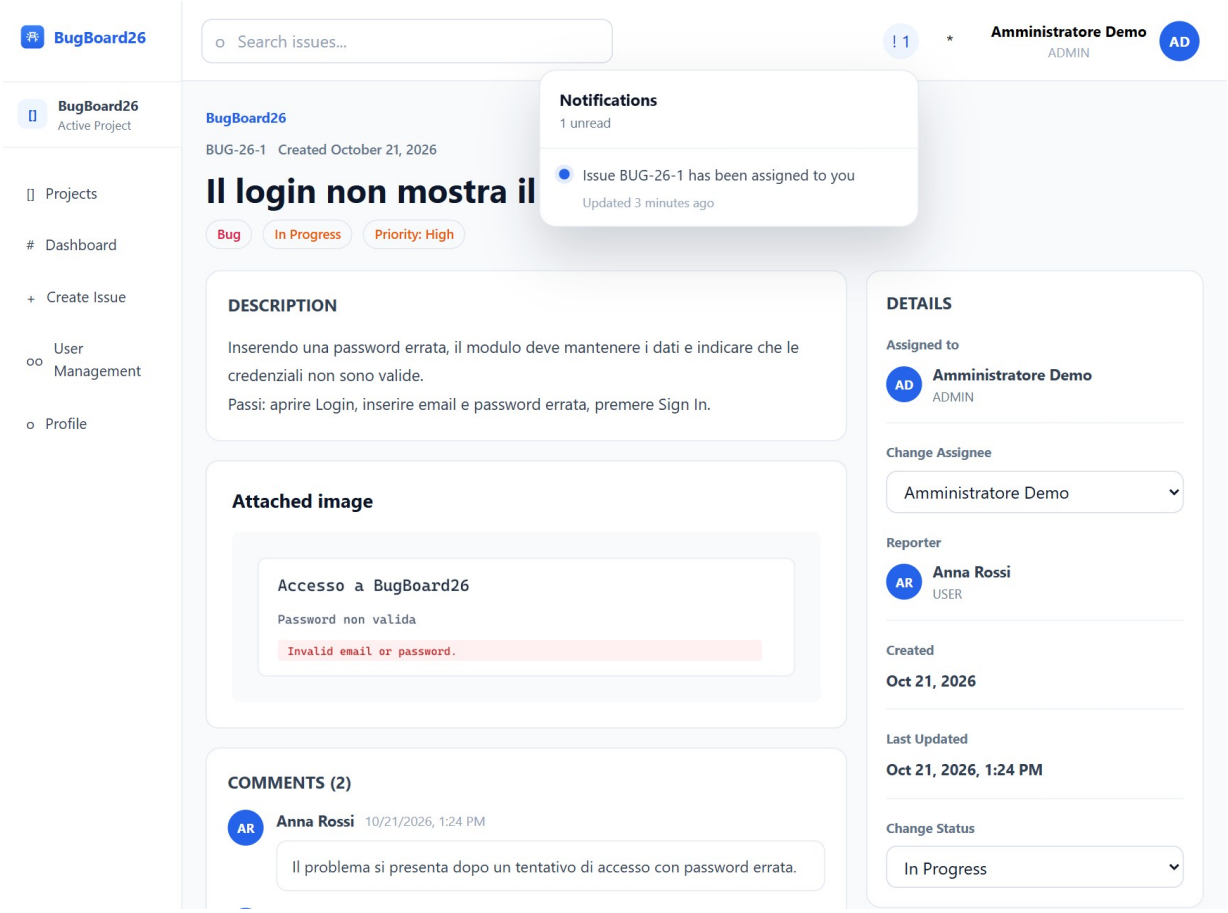


Figura 11 Pannello delle notifiche personali nella barra superiore

Il collegamento a una issue viene mostrato soltanto quando il destinatario può ancora accedervi. Una notifica di risoluzione destinata a un ex membro conserva il messaggio senza riaprire l'accesso al progetto.

6.7 Amministrazione e limiti da verificare

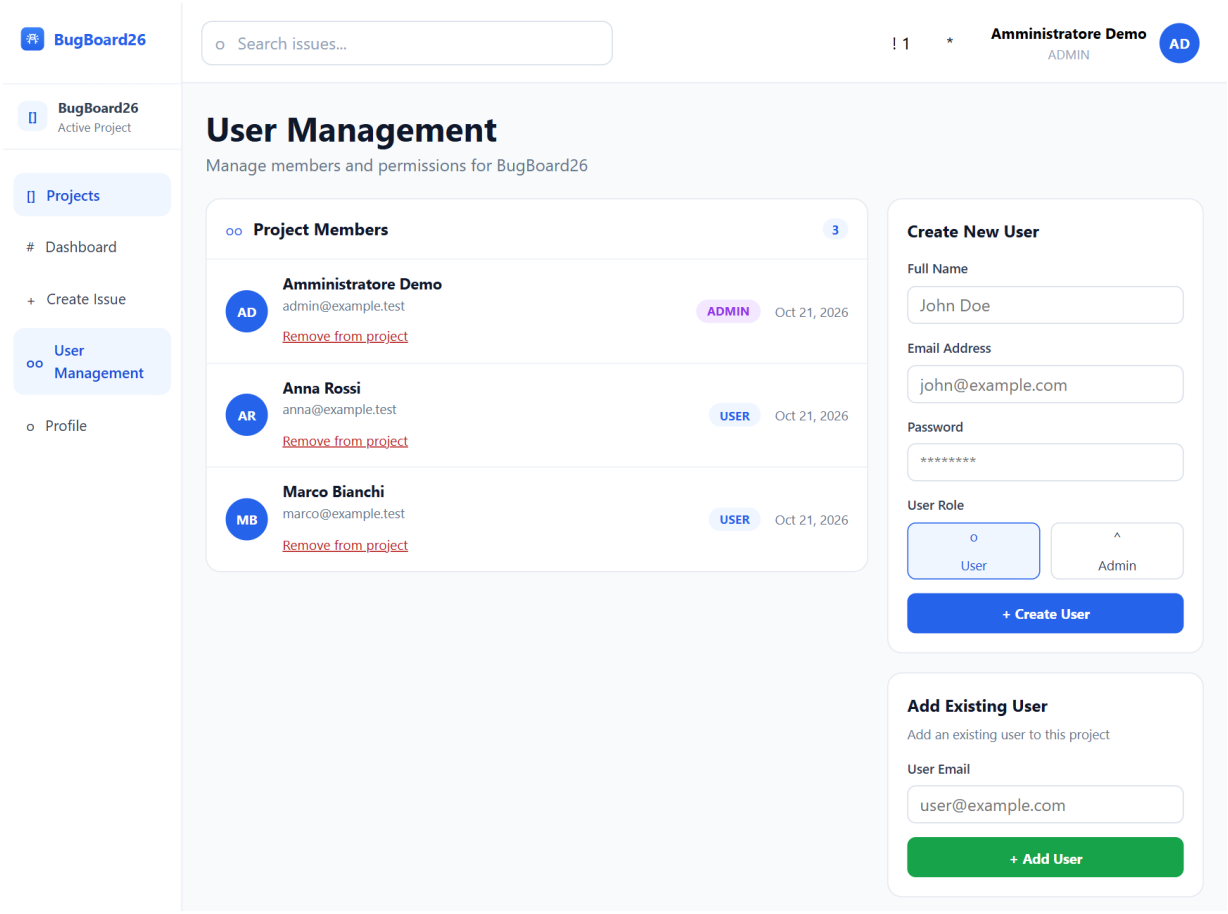


Figura 12 Gestione degli account e dei membri di progetto riservata agli amministratori

La pagina distingue creazione di un account e aggiunta di un account esistente al progetto. Il ruolo è globale e non diventa un ruolo differente per ciascun progetto. L'utente normale non dispone di questa pagina di amministrazione.

La struttura segue una lettura dall'alto verso il basso e mantiene controlli correlati vicini. Queste sono motivazioni del design, non risultati già misurati su utenti. La ricerca testuale in testata, Settings e Profile sono ancora elementi dimostrativi; la revisione dell'usabilità dovrà verificare se disabilitarli o renderne esplicito lo stato per non creare aspettative errate.

7 Moduli e diagrammi di design

7.1 Design del backend

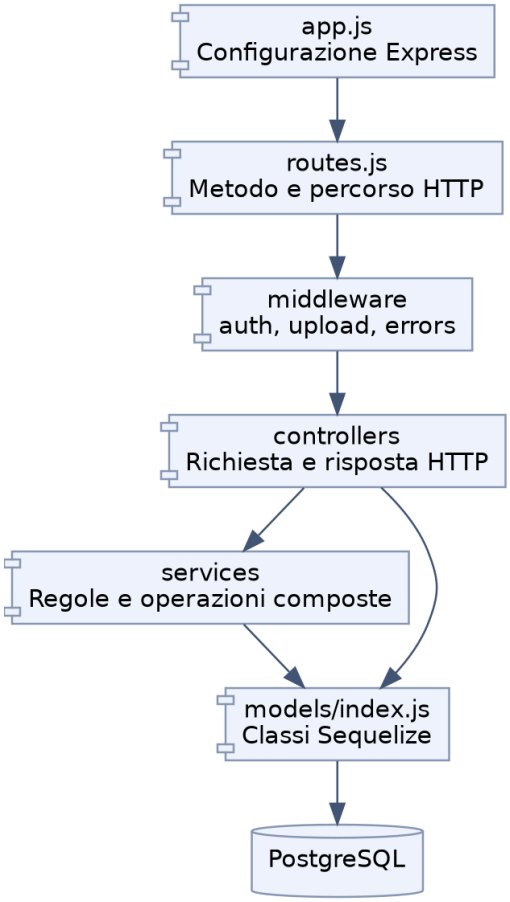


Figura 13 Dipendenze tra i moduli del backend

Nel backend JavaScript controller e servizi sono moduli con funzioni esportate. Il diagramma rappresenta pertanto componenti e dipendenze, non classi Java inesistenti. Le classi persistenti effettive sono User, Project, Membership, Issue, IssueImage, Comment e Notification, tutte definite in models/index.js attraverso Sequelize Model.

Percorso	Responsabilità concreta
src/app.js	Configura Express, parser JSON, CORS, rotte e gestione degli errori; non avvia la porta.
src/server.js	Verifica il database, prepara l'admin di sviluppo quando previsto e avvia HTTP.
src/routes.js	Associa metodo e percorso al controller e agli eventuali middleware.
src/controllers/	Legge body e parametri, coordina l'operazione e restituisce lo stato HTTP.
src/services/	Contiene autenticazione, controlli condivisi, immagini, notifiche e transazioni delle issue.
src/middleware/	Verifica il JWT, limita upload e accessi ADMIN e uniforma gli errori.
src/config/database.js	Prepara Sequelize usando variabili di ambiente, senza alterare lo schema.

7.2 Design del frontend

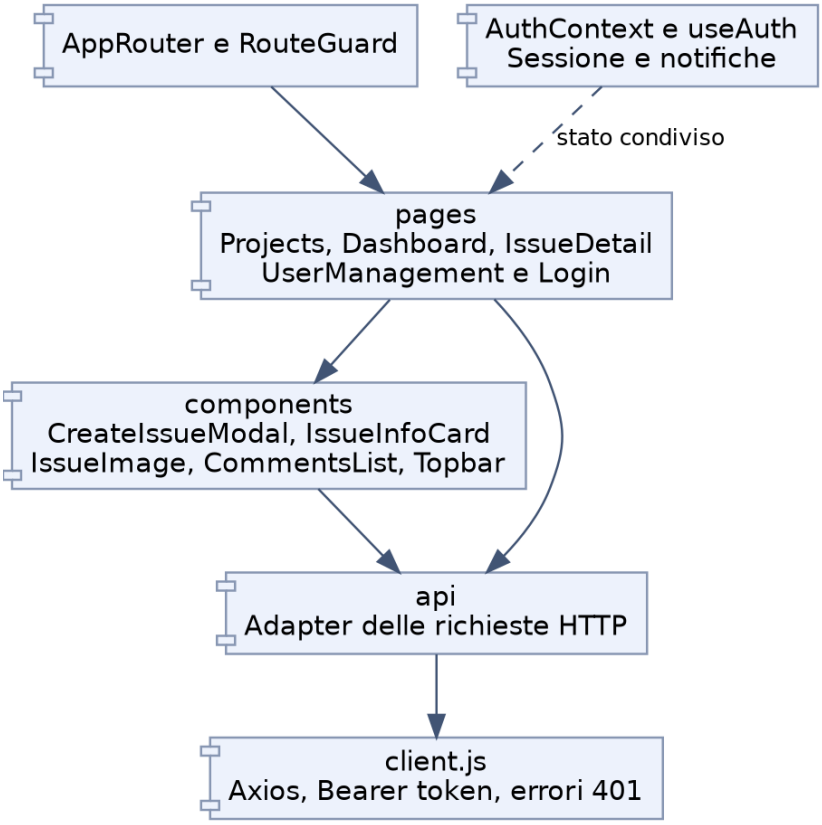


Figura 14 Componenti e moduli del frontend React

Le pagine caricano i dati e coordinano le azioni. I componenti presentano controlli riutilizzabili, mentre gli adapter in api preparano le richieste. AuthContext conserva la sessione condivisa e rende disponibili utente e notifiche con useAuth. RouteGuard protegge la navigazione; il backend rimane il punto che decide i permessi reali.

Lo stato della form e dei caricamenti è locale ai componenti. Quando una richiesta termina, React aggiorna le parti interessate. IssueImage scarica l'allegato separatamente per evitare che un errore dell'immagine impedisca di leggere descrizione e commenti.

7.3 API e formati delle risposte

Metodo e percorso con prefisso /api	Uso e permesso
POST /auth/login	Accesso pubblico con credenziali; restituisce token, scadenza e utente.
GET /users/me	Account autenticato. GET e POST /users sono riservate ad ADMIN.
GET /projects e GET /projects/{id}	Progetti visibili; POST /projects è riservata ad ADMIN.
GET /projects/{id}/members	Elenco membri per utenti autorizzati; POST e DELETE richiedono ADMIN.
GET /issues e GET /projects/{id}/issues	Liste limitate ai progetti accessibili.

Metodo e percorso con prefisso /api	Uso e permesso
POST /issues	Creazione JSON o multipart con immagine facoltativa; risposta 201.
GET /issues/{id} e GET /issues/{id}/image	Dettaglio e byte dell’allegato dopo il controllo di accesso.
PATCH /issues/{id}/assignee	Cambio membro assegnato oppure assigneeId null.
PATCH /issues/{id}/status	Cambio stato riservato all’assegnatario corrente.
GET e POST /issues/{id}/comments	Lettura e aggiunta dei commenti da parte degli utenti autorizzati.
GET /notifications e PATCH /notifications/{id}/read	Notifiche personali e marcatura letto.

Le liste usano items e totalCount. Le notifiche aggiungono unreadCount. Gli errori JSON contengono timestamp, status, code, message, path ed errors. Gli esiti principali sono 400 per dati non validi, 401 per autenticazione non valida, 403 per permesso negato, 404 per risorsa non disponibile, 409 per conflitto e 413 per file troppo grande.

7.4 Sequenza di autenticazione

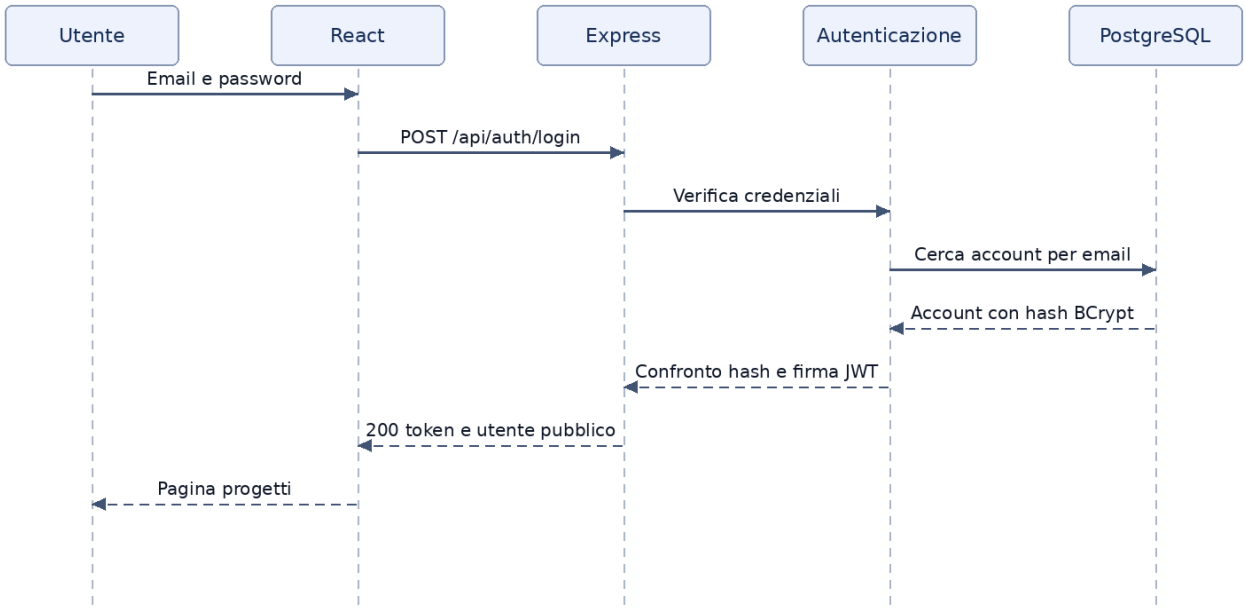


Figura 15 Sequenza di login e creazione del token

Il controller prepara il risultato HTTP, mentre il servizio di autenticazione confronta la password e prepara il token. Il database conserva soltanto l'hash della password. Nelle richieste successive il middleware verifica il token e ricarica l'account, prima di entrare nel controller.

7.5 Sequenza di creazione con immagine

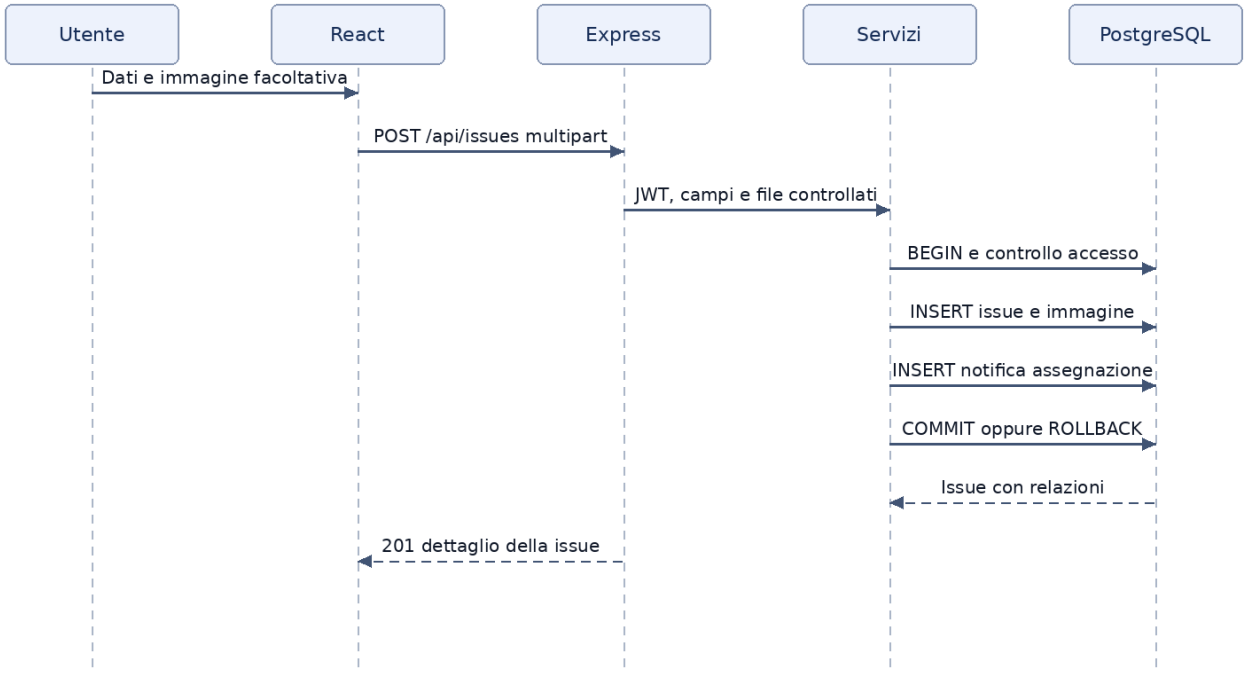


Figura 16 Creazione della issue con allegato nella stessa transazione

L'immagine viene controllata prima del salvataggio. Il servizio apre una transazione, verifica l'accesso al progetto e il destinatario, crea la issue e l'allegato e prepara le notifiche. In caso di errore il rollback impedisce di lasciare una issue parziale o un allegato senza issue.

7.6 Sequenza di risoluzione

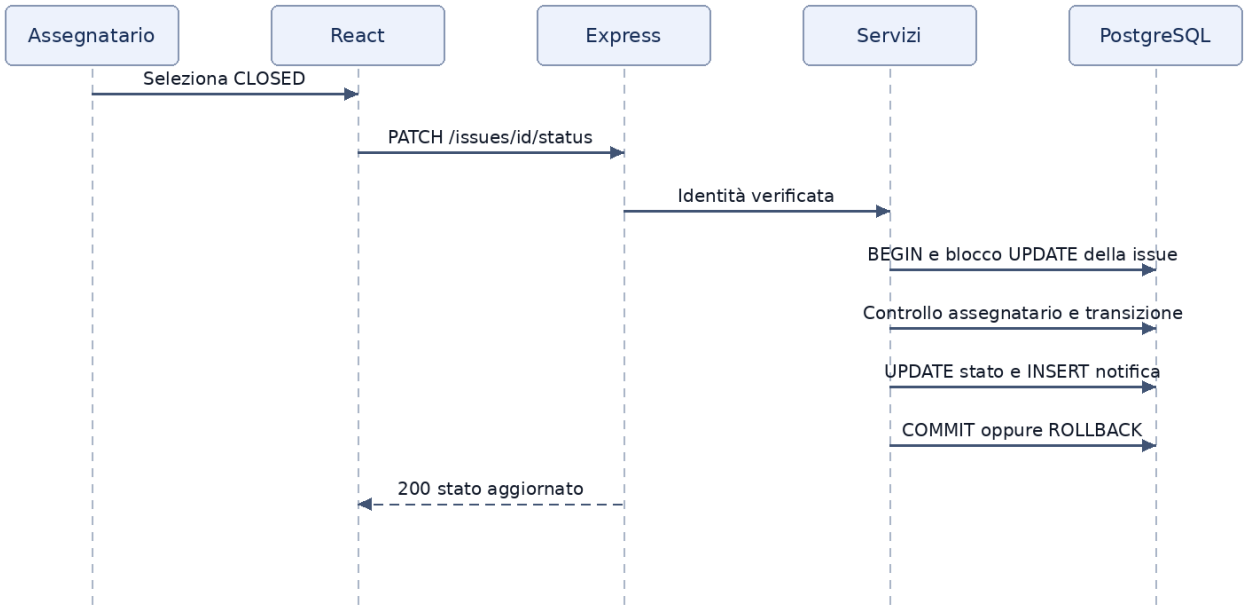


Figura 17 Cambio stato e notifica di risoluzione al reporter

Il controllo di assegnazione è eseguito sulla issue riletta nella transazione e prima di trattare una richiesta ripetuta come idempotente. Un amministratore non assegnato riceve quindi 403 anche se richiede lo stato corrente. Quando IN_PROGRESS diventa CLOSED, la notifica di risoluzione e l'aggiornamento di stato vengono confermati insieme.

8 Build e avvio del sistema

8.1 Backend e Dockerfile

Il Dockerfile del backend usa Node 24 Alpine, installa le dipendenze del lockfile con npm ci --omit=dev e copia sorgenti e script SQL. Il processo è eseguito con l'utente node. Le immagini non sono salvate nel filesystem del container, ma in PostgreSQL. La build e la produzione di un'immagine PNG con Sharp nel container sono state verificate.

8.2 Build del frontend

Il frontend usa gli script di Vite per sviluppo e build. npm run build produce dist con gli asset statici; npm run dev avvia il server di sviluppo, normalmente sulla porta 5173. Nel repository non è presente un Dockerfile frontend: il Compose attuale avvia PostgreSQL e backend, mentre il frontend viene avviato separatamente.

8.3 Docker Compose e configurazione

Compose espone il backend su 8081 e PostgreSQL sulla porta host configurata, prevista come 5433. Tra container PostgreSQL resta sulla porta 5432. Il volume postgres_data conserva i dati. Le variabili del backend sono documentate in backend/.env.example; il client può configurare VITE_API_BASE_URL e VITE_USE_MOCK_API.

Prima di utilizzare una configurazione già esistente, verificare la porta effettiva pubblicata dal container. Sul computer usato per lo sviluppo era presente anche un container mappato sulla porta host 5432: i file locali di ambiente devono riflettere la configurazione effettiva senza ricreare il database.

8.4 Configurazione e comandi

Dalla cartella principale, con Docker Desktop aperto, la prima configurazione locale segue questi passaggi PowerShell. La copia dei file di ambiente va eseguita solo se i file non esistono già.

```
docker compose -f .\infra\docker\docker-compose.yml up -d postgres
cd backend
if (-not (Test-Path .env)) { Copy-Item .env.example .env }
npm.cmd ci
npm.cmd run db:init
npm.cmd run dev
```

In un secondo terminale, dalla cartella principale:

```
npm.cmd --prefix frontend ci
npm.cmd --prefix frontend run dev
```

Il comando db:init deve essere eseguito anche aggiornando un database esistente alla versione con allegati. Il server non applica modifiche allo schema durante l'avvio. Le credenziali dell'admin iniziale dimostrativo sono descritte nel README e vanno sostituite in un ambiente esterno.

Per avviare anche il backend con Docker, eseguire dalla cartella principale:

```
docker compose -f .\infra\docker\docker-compose.yml build backend
docker compose -f .\infra\docker\docker-compose.yml run --rm backend npm run db:init
docker compose -f .\infra\docker\docker-compose.yml up -d
```

9 Strumento di versionamento

Il progetto è versionato con Git e dispone di un repository remoto su GitHub. I commit distinguono introduzione delle funzionalità, migrazione tecnologica e successivi allineamenti alla traccia. Il lockfile di ciascun sottoprogetto accompagna le modifiche alle dipendenze.

Commit	Contenuto
aa15294	Gestione di issue, commenti e notifiche nella versione iniziale.
3144a8d	Migrazione del backend a Express e Sequelize mantenendo PostgreSQL.
55fa6f9	Priorità facoltativa, stato TODO e notifiche di risoluzione.
1cf16c6	Cambio stato riservato al solo assegnatario.

La relazione comprende anche l'implementazione locale delle immagini e i nuovi test, successiva all'ultimo commit indicato. Prima della consegna occorre associare al documento il commit o lo ZIP definitivo che comprende tali modifiche.

9.1 Organizzazione del lavoro

Una modifica viene analizzata rispetto ai requisiti, implementata nei moduli interessati e verificata con controlli appropriati. Prima di un aggiornamento remoto si controllano le differenze e lo stato del branch. I file di ambiente personali, le dipendenze installate e gli output temporanei non devono essere inclusi nei commit.

9.2 Struttura del repository

```
BugBoard26/
  backend/       API, modelli, servizi, SQL e test HTTP
  frontend/      SPA React e test dei componenti
  infra/docker/  Compose e configurazione PostgreSQL
  docs/          Contratti API e diagrammi esistenti
  output/        Documenti esportati per la consegna
```

10 Qualità del codice

Le verifiche disponibili riguardano comportamento, compilazione e coerenza dei moduli. Non è configurato un servizio di analisi statica come SonarQube e non vengono presentati grafici di duplicazione, debito tecnico o percentuali di coverage non misurate.

10.1 Controlli effettivamente eseguiti

Controllo	Esito	Significato
Test HTTP backend	52 superati	Verifica API e dati su PostgreSQL reale isolato.
Test frontend	17 superati	Verifica componenti e preparazione delle richieste con Vitest e jsdom.
ESLint frontend	Superato	Controlla regole sintattiche e problemi segnalati dalla configurazione ESLint.
Build Vite	Superata	Conferma che gli asset di produzione possono essere generati.
Build e prova Docker	Superate	Verifica installazione backend ed elaborazione immagini nell'immagine Linux.
Acquisizione schermate	8 schermate reali	Verifica la visualizzazione dei percorsi principali con dati dimostrativi.

10.2 Scelte di qualità osservabili

La separazione tra app.js e server.js permette di avviare HTTP su una porta temporanea nei test. I controller accettano solo i campi previsti; reporter, autore e stato iniziale sono stabiliti dal backend. I servizi di accesso riuniscono i controlli condivisi. Gli errori sono convertiti in una forma JSON uniforme senza esporre query o password.

Le operazioni composte utilizzano transazioni e i test verificano gli effetti di errori deliberati nel salvataggio delle notifiche. Le immagini non sono inserite nei JSON delle liste e gli URL temporanei del browser vengono liberati. Queste misure sono controlli concreti; non costituiscono una certificazione completa di sicurezza o prestazioni.

10.3 Verifiche ancora da completare

Restano da misurare prestazioni sotto carico, accessibilità complessiva e risultati con utenti. La suite di test non fornisce attualmente un report di coverage percentuale. Il completamento di queste attività deve aggiungere dati misurati e condizioni di esecuzione, senza dedurli dal solo numero di test superati.

11 Testing automatico

11.1 Strategia e ambiente

La verifica automatica comprende 52 test backend e 17 test frontend, tutti superati nell'esecuzione del 21 ottobre 2026. I test backend in backend/test/api.test.js effettuano vere richieste HTTP verso Express e usano PostgreSQL. Uno schema con nome casuale separa i dati di ciascuna esecuzione; test-docker.ps1 prepara un database temporaneo ed elimina soltanto il container creato dal test.

Il backend è verificato prevalentemente attraverso test di integrazione: middleware, controller, servizi e database lavorano insieme. I test frontend usano Vitest, Testing Library e jsdom: simulano i componenti e sostituiscono le API quando serve controllare gli stati dell'interfaccia. Non costituiscono una suite automatica completa di navigazione end to end.

Le suite seguono preparazione, azione e verifica con asserzioni e fixture. Il backend usa node:test, il frontend Vitest. La traccia richiede un framework della famiglia xUnit: l'accettabilità formale degli strumenti scelti deve essere confermata ai fini della consegna; non viene dichiarata un'equivalenza già approvata dal docente.

11.2 Comandi per ripetere i test

```
powershell -ExecutionPolicy Bypass -File .\backend\test-docker.ps1
npm.cmd --prefix frontend test
npm.cmd --prefix frontend run lint
npm.cmd --prefix frontend run build
```

In alternativa al container temporaneo è possibile impostare TEST_DATABASE_URL su un PostgreSQL dedicato ai test ed eseguire npm.cmd test nella cartella backend. Non va utilizzato un database di produzione. La suite genera e rimuove soltanto lo schema casuale di propria competenza.

11.3 Test selezionati del cambio di stato

Metodo esaminato: changeStatus in backend/src/services/issues.js. La funzione verifica l'accesso alla issue, blocca la riga nella transazione e controlla assegnatario e avanzamento. La verifica comprende classi di equivalenza per l'identità e per lo stato, oltre a idempotenza e fallimenti di persistenza.

Classe o condizione	Dati o azione	Risultato atteso e verificato
Assegnatario valido	TODO a IN_PROGRESS, poi CLOSED	200, stato persistente e notifica di risoluzione.
Reporter non assegnato	Richiesta di avanzamento	403 NOT_ISSUE_ASSIGNEE e nessuna modifica.
ADMIN non assegnato	Anche se membro del progetto	403; il ruolo non sostituisce l'assegnazione.
Nessun assegnatario	assigneeId NULL	403 prima della modifica.
Transizione non ammessa	TODO a CLOSED oppure ritorno a TODO	409 INVALID_STATUS_TRANSITION.
Richiesta ripetuta	Stesso stato richiesto dall'assegnatario	200 senza nuove notifiche.
Riassegnazione	Il precedente assegnatario riprova	403; il nuovo assegnatario può procedere.
Concorrenza	Due richieste identiche	Un solo avanzamento e nessuna notifica duplicata.
Scrittura fallita	Errore simulato in Notification.create	500; stato e data progetto tornano alla situazione precedente.

Questi test verificano anche che le richieste rifiutate non cambino assegnatario, date o conteggi delle notifiche. Il controllo resta nel backend, quindi vale anche per richieste inviate senza utilizzare l'interfaccia React.

11.4 Test selezionati delle immagini

Il percorso controllato comprende uploadImage, prepareImage, creazione in transazione e GET /issues/{id}/image. I file validi di prova sono generati in memoria. Le verifiche non si limitano al nome o all'estensione, ma controllano i byte restituiti e il formato decodificato.

Caso	Valore o condizione	Risultato verificato
Formato ammesso	PNG, JPEG, WebP validi	201, allegato persistente e download autenticato con MIME corretto.
Allegato assente	Richiesta JSON o multipart senza file	201, image null e nessuna riga issue_images.
Dimensione oltre il limite	5242881 byte	413 IMAGE_TOO_LARGE e nessuna issue creata.
Contenuto falso	Testo, SVG mascherato, file vuoto o troncato	400 INVALID_IMAGE senza salvataggi parziali.
Tipo dichiarato non coerente	Byte PNG dichiarati come JPEG	400 INVALID_IMAGE.
Troppi pixel	4001 per 4000 pixel	Rifiuto: supera il limite di 16000000 pixel.
Metadati privati	Immagine con dati EXIF	I metadati non sono presenti nell'immagine salvata.
Accesso negato	Token assente, esterno o membro rimosso	401 o 403; i byte non vengono restituiti.
Persistenza	Due riavvii del server e db:init ripetuto	Gli stessi byte dell'immagine restano disponibili.
Errore nella transazione	Fallimento della notifica dopo la creazione	Rollback anche della issue e dell'immagine.

11.5 Test frontend

File	Numero	Controlli
CreateIssueModal.test.jsx	7	Creazione senza priorità e immagine; scelta e rimozione file; tipo errato, file vuoto o oltre limite; conservazione dei dati dopo un errore.
IssueImage.test.jsx	3	Caricamento del Blob, rilascio dell'URL temporaneo, gestione dell'errore con Retry e chiusura durante il download.
IssueInfoCard.test.jsx	4	Controllo dello stato abilitato per l'assegnatario e disabilitato per altri utenti, issue non assegnate e issue chiuse.
issuesApi.test.js	3	Compatibilità JSON senza immagine, richiesta multipart con file e download binario tramite client autenticato.

I test dei componenti verificano il comportamento osservabile del form e dei messaggi, senza riprodurre tutte le funzioni interne. I controlli più rilevanti sui permessi restano coperti dai test HTTP con database reale. La build e il lint sono verifiche aggiuntive e non sono contati come test funzionali.

11.6 Tracciabilità e copertura funzionale

Requisiti	Evidenze nella suite
RF01-RF02	Login valido/errato, email normalizzata, JWT alterato o scaduto, account sicuro, unicità e permessi ADMIN.
RF03	Creazione progetto con membro iniziale, visibilità, aggiunta duplicata e revoca immediata.
RF04-RF06	Quattro tipi, priorità facoltativa, dati controllati dal server e otto test dedicati al percorso immagini.
RF07	Statistiche persistenti; filtri osservati nelle schermate. I filtri combinati richiedono ancora una prova specifica dell'interfaccia.
RF08-RF10	Assegnatario membro, riassegnazione, nessun assegnatario, commenti, transizioni e autorizzazione esclusiva.
RF11-RF12	Deduplicazione, notifica di risoluzione al reporter, privacy, revoca, lettura idempotente e rollback.

La copertura descritta è funzionale e parziale. Non viene dedotta una percentuale di copertura del codice. I test non sostituiscono le prove di usabilità e non dimostrano da soli il comportamento sotto carico elevato.

12 Valutazione della usabilità

La valutazione con utenti e i questionari devono ancora essere svolti. Questo capitolo definisce il protocollo, la checklist e le schede da utilizzare per raccogliere i risultati. Non sono presenti partecipanti, voti o medie attribuiti a prove non eseguite.

12.1 Review sistematica con checklist

La review dovrà essere svolta da persone che non abbiano appena implementato il percorso esaminato. Per ogni voce si registra Conforme, Parziale, Non conforme oppure Non applicabile, con schermata o passo del task come evidenza. Le criticità sono distinte in lieve, rilevante e bloccante in base all'effetto sul completamento del compito.

Area	Domanda di verifica	Esito e note
Coerenza	Etichette, posizione delle azioni e significato di colori e badge sono uniformi tra le pagine?	Da compilare
Visibilità dello stato	Caricamento, creazione, errori e cambio stato sono riconoscibili senza ripetere l'azione?	Da compilare
Chiusura dei compiti	Dopo una creazione o un invio è chiaro che l'operazione è terminata e dove si trova il risultato?	Da compilare
Prevenzione degli errori	Campi obbligatori, priorità facoltativa e limiti del file sono comprensibili prima dell'invio?	Da compilare
Recupero	Un invio fallito conserva testo e file? Retry e annullamento sono individuabili?	Da compilare
Controllo dell'utente	I controlli disabilitati spiegano il motivo, soprattutto per assegnazione e stato?	Da compilare
Carico di memoria	Progetto, reporter, assegnatario e stato sono visibili nel contesto della issue?	Da compilare
Accessibilità e tastiera	Focus, etichette, contrasto, ordine di tabulazione e dialoghi sono utilizzabili senza mouse?	Da compilare
Responsive	Su schermi piccoli i campi, le notifiche e i pulsanti restano visibili e raggiungibili?	Da compilare
Aspettative	I controlli dimostrativi non attivi sono riconoscibili e non inducono a un percorso senza esito?	Da compilare

12.2 Piano delle prove con utenti

Si propone di coinvolgere cinque utenti reali o potenziali, numero indicativo da adattare alle disponibilità del gruppo. I profili dovrebbero includere persone che conoscono strumenti di gestione delle issue e persone che li usano raramente. Gli account di prova sono predisposti dal gruppo e i dati non devono contenere informazioni personali reali.

Ogni sessione prevede una breve introduzione, l'esecuzione dei task senza suggerire i comandi da usare, l'osservazione delle interazioni e un questionario finale. Il partecipante può ragionare ad alta voce; l'osservatore annota ostacoli e interventi. Non si registra audio, video o altri dati personali senza un accordo esplicito del partecipante.

Task	Obiettivo da comunicare al partecipante	Criterio di completamento
T01	Accedi all'account di prova e trova il progetto indicato.	Progetto corretto aperto senza assistenza.
T02	Segnala un bug con descrizione e schermata, lasciando la priorità non specificata.	Issue creata con immagine e priority null.
T03	Trova nella dashboard soltanto i bug ancora da lavorare.	Filtri coerenti e issue attesa individuata.
T04	Assegna la issue al membro indicato e aggiungi un aggiornamento.	Assegnazione corretta e commento persistente.
T05	Con l'account assegnatario, prendi in carico la issue e poi chiudila.	TODO a IN_PROGRESS a CLOSED senza tentativi impropri.
T06	Con l'account del reporter, individua la notifica di risoluzione e leggila.	Notifica corretta trovata e marcata letta.
T07	Prova a selezionare un file non ammesso e correggi la scelta.	Messaggio compreso e file valido selezionato senza perdere la descrizione.

12.3 Dati da raccogliere

Per ogni task si annotano successo o fallimento, tempo in secondi, errori osservati, richieste di aiuto e commenti del partecipante. Un successo assistito va distinto da un successo autonomo. I tempi vanno rilevati con lo stesso criterio per tutti: dal termine della consegna al raggiungimento dell'obiettivo oppure all'abbandono.

ID	Esperienza con issue tracker	Ruolo provato	Note
P01	Da rilevare	Da assegnare	
P02	Da rilevare	Da assegnare	
P03	Da rilevare	Da assegnare	
P04	Da rilevare	Da assegnare	
P05	Da rilevare	Da assegnare	

12.4 Scheda di osservazione

Partecipante: _____ Data: _____ Osservatore: _____

Task	Esito autonomo o assistito	Tempo s	Errori e osservazioni
T01	Da rilevare		
T02	Da rilevare		
T03	Da rilevare		
T04	Da rilevare		
T05	Da rilevare		
T06	Da rilevare		
T07	Da rilevare		

Osservazioni trasversali: _____

Interventi dell'osservatore: _____

12.5 Questionario finale

Per ogni affermazione indicare un valore da 1 a 5: 1 significa per niente d'accordo, 3 indica una posizione intermedia, 5 significa completamente d'accordo. È un questionario specifico del progetto e non produce un punteggio SUS standardizzato.

ID	Affermazione	Risposta da 1 a 5
Q01	Ho capito come trovare il progetto e le sue issue.	
Q02	La creazione della issue distingue chiaramente campi obbligatori e facoltativi.	
Q03	Sono riuscito a scegliere e visualizzare l'immagine senza difficoltà.	
Q04	I filtri mi hanno aiutato a trovare le segnalazioni pertinenti.	
Q05	Ho capito la differenza tra reporter e assegnatario.	
Q06	Era chiaro chi poteva cambiare lo stato e quale passaggio era consentito.	
Q07	I messaggi di errore mi hanno aiutato a correggere il problema.	
Q08	Le notifiche mi hanno aiutato a capire gli aggiornamenti della issue.	

Quale passaggio è stato meno chiaro? _____

Quale modifica renderebbe l'applicazione più semplice? _____

12.6 Risultati e conclusioni da integrare

Al termine delle sessioni, questa sezione dovrà riportare numero e profili effettivi dei partecipanti, ambiente usato, task svolti e risultati osservati. Il tasso di completamento va calcolato sui task effettivamente tentati; i successi assistiti vanno riportati separatamente. Per i tempi si possono confrontare mediana e intervallo senza attribuire significato statistico a un campione limitato.

Le risposte al questionario possono essere riassunte per domanda insieme alle osservazioni qualitative. Eventuali medie devono indicare quanti valori sono disponibili. Le conclusioni dovranno collegare ogni criticità a una modifica proposta e, se possibile, a una successiva verifica. Fino alla raccolta dei dati non si può affermare che la valutazione di usabilità sia stata completata.

Riferimenti

[R1] React, Quick Start. https://react.dev/learn

[R2] Express, Routing e middleware. https://expressjs.com/en/guide/routing/

[R3] Sequelize 6, Transactions. https://sequelize.org/docs/v6/other-topics/transactions/

[R4] PostgreSQL 17, Constraints. https://www.postgresql.org/docs/17/ddl-constraints.html

[R5] Express, Multer. https://expressjs.com/en/resources/middleware/multer/

[R6] Sharp, Constructor e Output options. https://sharp.pixelplumbing.com/api-constructor/

Codice del progetto BugBoard26: backend/src, backend/sql, backend/test, frontend/src e frontend/test.

Documenti interni: README, contratti API delle milestone e configurazione Docker. In caso di differenze con documenti precedenti, la presente relazione descrive la versione corrente con allegati.